

CSE 713: Artificial Intelligence

Search Algorithms — Complete Model Answers

University of Chittagong — 7th Semester B.Sc. (Engg.) Examination

Covers Years: 2024, 2023, 2022, 2021, 2020 — Topics: UCS, BFS, DFS, Greedy, A*,
IDDFS, IDA*

Contents

1 Examination 2024 — Q2 (9 Marks)	2
1.1 Problem Data	2
1.2 Part a(i) — State-Space Graph [1 mark]	2
1.3 Part a(ii) — Search Algorithm Traces [5 marks]	3
1.4 Part b — Problem Formulations [3 marks]	8
2 Examination 2023 — Q2 (9 Marks)	10
2.1 Problem Data	10
2.2 Part a — Problem Formulations [2 marks]	10
2.3 Part b(i) — State-Space Diagram [0.5 mark]	11
2.4 Part b(ii) — Search Algorithm Traces [4.5 marks]	12
2.5 Part c — IDDFS: Synthesising Benefits of BFS and DFS [2 marks]	15
3 Examination 2022	18
3.1 Section A — Q.A-4(a): DFS vs BFS [2 marks]	18
3.2 Section B — Q.B-4(b): Graph Trace [3 marks]	21
4 Examination 2021	22
4.1 Q2(a) — State Space Search Problem [0.75 mark]	22
4.2 Q2(b) — Problem Formulations [3 marks]	22
4.3 Q2(c) — Graph Traces [5 marks]	24
4.4 Q3(b) — DFID vs IDA* [1.75 marks]	27
5 Examination 2020	30
5.1 Q2(a) — Formulating a Search Problem [1.75 marks]	30
5.2 Q2(b) — Problem Formulations [2 marks]	31
5.3 Q2(c) — Graph Traces [5 marks]	32
5.4 Q3(b) — IDDFS Combines BFS + DFS [2 marks]	33
5.5 Q3(c) — BFS Admissibility [1.75 marks]	35
5.6 Q7(b) — A* Heuristic: Underestimate vs Overestimate [2 marks]	36

Examination 2024 — Q2 (9 Marks)

Problem Data

States: A (start), B, C, D, E, F, H, J, K, G (goal). All roads bidirectional.

Table 1: Road Connections and Step Costs

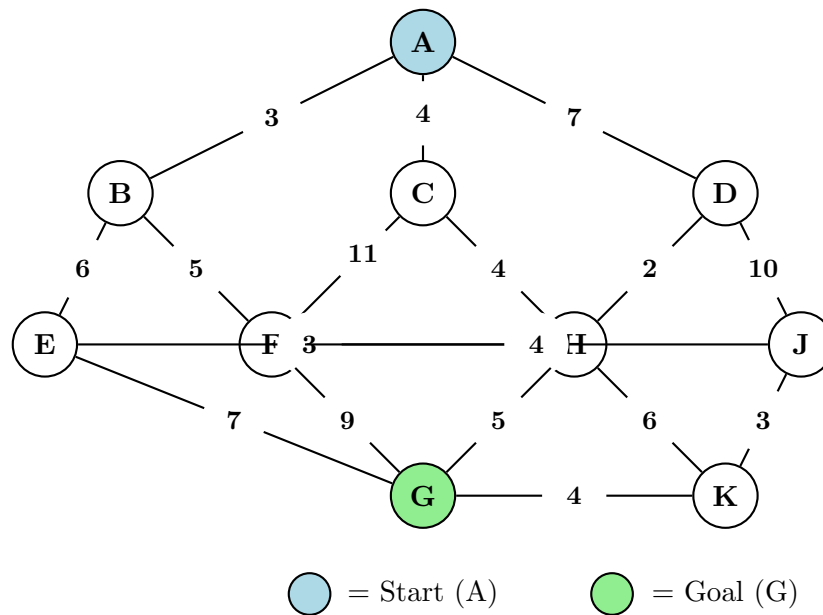
From	To	Cost
A	B	3
A	C	4
A	D	7
B	E	6
B	F	5
C	F	11
C	H	4
D	H	2
D	J	10
E	G	7
E	H	3
F	G	9
F	J	4
H	G	5
H	K	6
J	K	3
K	G	4

Table 2: Heuristic Estimates $h(n)$ to Goal G

Node	$h(n)$
A	11
B	10
C	7
D	6
E	6
F	6
H	4
J	5
K	3
G	0

Part a(i) — State-Space Graph [1 mark]

Question: Using Table 1, draw the state-space graph clearly labelling all nodes and edge costs.



Note: The edge E–H is curved upward and F–J is curved downward to avoid visual ambiguity where nodes share the same y -coordinate. All edges are **bidirectional** (undirected).

Part a(ii) — Search Algorithm Traces [5 marks]

Question: Show search trees for (1) UCS, (2) Greedy Best-First, (3) A*, (4) IDDFS from A to G. At every step: show node being expanded and fringe contents in order of selection. For IDDFS: show depth limit per iteration. Ties broken alphabetically.

Algorithm 1 — Uniform-Cost Search (UCS)

Key idea: Always expand the node with the **lowest path cost** $g(n)$ from the start. The fringe is a **min-priority queue sorted by** $g(n)$. A node is closed (never re-expanded) once popped. If a *cheaper* path to a fringe node is found, its cost is updated (replaced).

Fringe notation: Node[g] sorted ascending by g .

Step	Node Expanded	$g(n)$	Fringe after expansion (sorted by g)
1	A	0	B[3], C[4], D[7] Neighbours of A: B($g=3$), C($g=4$), D($g=7$).
2	B	3	C[4], D[7], F[8], E[9] A already closed. $B \rightarrow F$: $g=8$; $B \rightarrow E$: $g=9$. Both new.
3	C	4	D[7], F[8], H[8], E[9] $C \rightarrow F$: $g=15 > 8$ already in fringe $\Rightarrow$ keep F[8]. $C \rightarrow H$: $g=8$ new.
4	D	7	F[8], H[8], E[9], J[17] $D \rightarrow H$: $g=9 > 8$ in fringe $\Rightarrow$ keep H[8]. $D \rightarrow J$: $g=17$ new. Tie F,H: F before H (alphabetical).
5	F	8	H[8], E[9], J[12], G[17] $F \rightarrow G$: $g=17$ new. $F \rightarrow J$: $g=12 < 17$ in fringe $\Rightarrow$ update J[12].
6	H	8	E[9], J[12], G[13], K[14] $H \rightarrow E$: $g=11 > 9$ in fringe $\Rightarrow$ keep E[9]. $H \rightarrow G$: $g=13 < 17 \Rightarrow$ update G[13]. $H \rightarrow K$: $g=14$ new.
7	E	9	J[12], G[13], K[14] $E \rightarrow G$: $g=16 > 13$ in fringe $\Rightarrow$ keep G[13].
8	J	12	G[13], K[14] $J \rightarrow K$: $g=15 > 14$ in fringe $\Rightarrow$ keep K[14].
9	G	13	GOAL FOUND

UCS Solution: $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$ **Total cost:** $4 + 4 + 5 = 13$
 Nodes expanded (in order): A, B, C, D, F, H, E, J, G (9 nodes total)
 UCS **guarantees the optimal path** because it always expands the cheapest unexplored node, never re-expands a closed node, and path costs are non-negative.

Algorithm 2 — Greedy Best-First Search

Key idea: Always expand the node with the **lowest heuristic estimate** $h(n)$ to the goal. The fringe is sorted by $h(n)$ (ascending). Path cost $g(n)$ is *completely ignored*. Greedy is fast but not guaranteed to find the optimal path.

Fringe notation: Node[h] sorted ascending by h .

Step	Node Expanded	$h(n)$	Fringe after expansion (sorted by h)
1	A	11	D[6], C[7], B[10] Neighbours B($h=10$), C($h=7$), D($h=6$). Sorted by h : D, C, B.
2	D	6	H[4], J[5], C[7], B[10] D's neighbours excl. A: H($h=4$), J($h=5$).
3	H	4	G[0], K[3], J[5], E[6], C[7], B[10] H's neighbours excl. D: C($h=7$, already in fringe—skip), E($h=6$), G($h=0$), K($h=3$).
4	G	0	GOAL FOUND

Greedy Solution: $A \xrightarrow{7} D \xrightarrow{2} H \xrightarrow{5} G$ **Total cost:** $7 + 2 + 5 = 14$ (not optimal!)

Nodes expanded: A, D, H, G (only 4 nodes — very fast)

Greedy expanded just 4 nodes versus UCS's 9, but found a **suboptimal path** (cost 14 vs optimal 13). It followed the heuristic blindly: D had the lowest h from A, leading it down the D–H–G route instead of the cheaper A–C–H–G route.

Algorithm 3 — A* Search

Key idea: Expand the node with the **lowest evaluation function** $f(n) = g(n) + h(n)$, where $g(n)$ = exact cost from start and $h(n)$ = heuristic estimate to goal. The fringe is sorted by $f(n)$ (ascending). Ties in f broken alphabetically. If a cheaper path to a fringe node is found, update its entry.

Fringe notation: Node[g, h, f] sorted ascending by f .

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	A	0	11	11	C[4,7, 11], B[3,10, 13], D[7,6, 13] Tie B,D at $f=13$: B first.
2	C	4	7	11	H[8,4, 12], B[3,10, 13], D[7,6, 13], F[15,6, 21] $C \rightarrow H$: $g=8, f=12$ new. $C \rightarrow F$: $g=15, f=21$ new.
3	H	8	4	12	B[3,10, 13], D[7,6, 13], G[13,0, 13], E[11,6, 17], K[14,3, 17], F[15,6, 21] $H \rightarrow D$: $f=16 > 13$, keep. $H \rightarrow G$: $f=13$ new. $H \rightarrow E$: $f=17$ new. $H \rightarrow K$: $f=17$ new. Tie B,D,G at $f=13$: B first .
4	B	3	10	13	D[7,6, 13], G[13,0, 13], F[8,6, 14], E[9,6, 15], K[14,3, 17] $B \rightarrow F$: $g=8, f=14 < 21 \Rightarrow$ update F. $B \rightarrow E$: $g=9, f=15 < 17 \Rightarrow$ update E. Tie D,G at $f=13$: D first .
5	D	7	6	13	G[13,0, 13], F[8,6, 14], E[9,6, 15], K[14,3, 17], J[17,5, 22] $D \rightarrow J$: $g=17, f=22$ new.
6	G	13	0	13	GOAL FOUND

A* Solution: $A \xrightarrow{4} C \xrightarrow{4} H \xrightarrow{5} G$ **Total cost:** $4 + 4 + 5 = 13$ (optimal!)
 Nodes expanded: A, C, H, B, D, G (6 nodes — fewer than UCS's 9)
 A* is both **complete** and **optimal** when the heuristic is *admissible* ($h(n) \leq h^*(n)$, i.e. it never overestimates). It is more efficient than UCS because $h(n)$ guides the search toward promising nodes, pruning unpromising directions early.

Algorithm 4 — Iterative Deepening DFS (IDDFS)

Key idea: Perform DFS with a depth cutoff $L = 0, 1, 2, \dots$, restarting from the root each time, until the goal is found. The “fringe” is a **stack** (LIFO). Children are pushed in *reverse* alphabetical order so they are *popped* (and expanded) in alphabetical order. Nodes on the **current path** (ancestors) are not revisited to avoid infinite loops.

Stack notation: Leftmost entry = top of stack (next to be popped). Depth shown in parentheses.

Iteration $L = 0$:

Step	Action	Stack (top $\leftarrow$ left)
1	Pop A (d=0). $A \neq G$. At depth limit — cannot expand.	[]
<i>No solution at $L = 0$.</i>		

Iteration $L = 1$:

Step	Action	Stack
1	Pop A (d=0). Expand: push D,C,B (B on top).	B(1), C(1), D(1)
2	Pop B (d=1). $B \neq G$. At depth limit.	C(1), D(1)
3	Pop C (d=1). $C \neq G$. At depth limit.	D(1)
4	Pop D (d=1). $D \neq G$. At depth limit.	[]
<i>No solution at $L = 1$.</i>		

Iteration $L = 2$:

Step	Action	Stack
1	Pop A (d=0). Expand: B,C,D.	B(1), C(1), D(1)
2	Pop B (d=1). Expand (excl. A): E,F. Push F,E.	E(2), F(2), C(1), D(1)
3	Pop E (d=2). $E \neq G$. At limit.	F(2), C(1), D(1)
4	Pop F (d=2). At limit.	C(1), D(1)
5	Pop C (d=1). Expand (excl. A): F,H. Push H,F.	F(2), H(2), D(1)
6	Pop F (d=2). At limit.	H(2), D(1)
7	Pop H (d=2). At limit.	D(1)
8	Pop D (d=1). Expand (excl. A): H,J. Push J,H.	H(2), J(2)
9	Pop H (d=2). At limit.	J(2)
10	Pop J (d=2). At limit.	[]
<i>No solution at $L = 2$.</i>		

Iteration $L = 3$:

Step	Action	Stack
1	Pop A (d=0). Expand: B,C,D.	B(1), C(1), D(1)
2	Pop B (d=1). Expand (excl. A): E,F.	E(2), F(2), C(1), D(1)
3	Pop E (d=2). Expand (excl. B): G,H. Push H first then G (G on top).	G(3), H(3), F(2), C(1)
4	Pop G (d=3). $G = \text{GOAL}$.	GOAL FOUND

IDDFS Solution: $A \xrightarrow{3} B \xrightarrow{6} E \xrightarrow{7} G$ **Depth: 3** **Cost:** $3 + 6 + 7 = 16$

IDDFS found the solution at depth-limit $L = 3$ in just 4 node expansions during the final iteration.

Important distinction: IDDFS finds the *shallowest* path (fewest hops), **not** the lowest-cost path. The optimal path $A \rightarrow C \rightarrow H \rightarrow G$ also has depth 3, but alphabetical DFS expansion reaches G via $B \rightarrow E \rightarrow G$ first. For non-uniform step costs, IDDFS is **not optimal**. It is however **complete** — it will always find a solution if one exists.

Why IDDFS over BFS? IDDFS uses only $O(bL)$ space (a single stack), whereas BFS requires $O(b^L)$ space to store all frontier nodes. The repeated work is a constant factor overhead.

Algorithm Comparison Summary:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
UCS	$A \rightarrow C \rightarrow H \rightarrow G$	13	9 (all iterations)	Yes
Greedy	$A \rightarrow D \rightarrow H \rightarrow G$	14	4	No
A*	$A \rightarrow C \rightarrow H \rightarrow G$	13	6	Yes
IDDFS	$A \rightarrow B \rightarrow E \rightarrow G$	16	4 (final iter.)	No (non-uniform costs)

Part b — Problem Formulations [3 marks]

Question: Give the initial state, goal test, operators, and path cost function for each problem.

(i) Robot Vacuum Cleaning Problem (Indoor Environment)

State: A pair (l, D) where l is the robot's current room and $D \subseteq \{\text{all rooms}\}$ is the set of dirty rooms. *Example:* (Kitchen, {Bedroom, Lounge}).

Initial state: (start room, {all currently dirty rooms}).

Goal test: $D = \emptyset$ (no dirty rooms remain).

Operators:

1. $\text{Move}(r_i \rightarrow r_j)$: Move from room r_i to adjacent room r_j (changes location; D unchanged).
2. $\text{Suck}()$: Vacuum the current room (removes current room from D if dirty).

Static obstacles (furniture) constrain which rooms are adjacent.

Path cost: Total energy used: each Move costs c_{move} energy units; each Suck costs c_{suck} units. Goal: minimise total energy.

Informed vs. Uninformed: **Informed search (A*) is more suitable.** A strong heuristic $h(n) = |D| \times \bar{d}$ (number of remaining dirty rooms multiplied

by average travel cost between rooms) estimates remaining work without overestimating. With many rooms, uninformed BFS/UCS must explore exponentially many (l, D) state combinations blindly. A* with a good heuristic dramatically prunes this search space.

(ii) Water Jug Problem (5L and 3L Jugs)

State: A pair (x, y) where $x \in \{0, 1, 2, 3, 4, 5\}$ is water (litres) in the 5L jug and $y \in \{0, 1, 2, 3\}$ is water in the 3L jug.

Initial state: $(0, 0)$ — both jugs empty.

Goal test: $x = 4$ (exactly 4 litres in the 5L jug). The 3L jug cannot hold 4L, so only $x = 4$ satisfies the goal.

Operators:

1. **Fill 5L:** $(x, y) \rightarrow (5, y)$
2. **Fill 3L:** $(x, y) \rightarrow (x, 3)$
3. **Empty 5L:** $(x, y) \rightarrow (0, y)$
4. **Empty 3L:** $(x, y) \rightarrow (x, 0)$
5. **Pour 5L $\rightarrow$ 3L:** $(x, y) \rightarrow (x-d, y+d)$ where $d = \min(x, 3-y)$
6. **Pour 3L $\rightarrow$ 5L:** $(x, y) \rightarrow (x+d, y-d)$ where $d = \min(y, 5-x)$

Path cost: Number of actions performed (1 per action), minimised to find the shortest sequence.

State space: **FINITE.** The 5L jug holds integer amounts $\{0, 1, 2, 3, 4, 5\}$ (6 values) and the 3L jug holds $\{0, 1, 2, 3\}$ (4 values), giving at most $6 \times 4 = 24$ states. Not all are reachable, but the space is strictly bounded.

One valid solution path (6 steps):

$$(0, 0) \xrightarrow{\text{Fill 5L}} (5, 0) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (2, 3) \xrightarrow{\text{Empty 3L}} (2, 0) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (0, 2) \xrightarrow{\text{Fill 5L}} (5, 2) \xrightarrow{\text{Pour 5} \rightarrow \text{3}} (4, 2)$$

Examination 2023 — Q2 (9 Marks)

Problem Data

Table 1: State Transitions and Step

Costs		
State	Next	Cost
A	B	4
A	C	1
B	D	3
B	E	8
C	C	0
C	D	2
C	F	6
D	C	2
D	E	4
E	G	2
F	G	8

Table 2: Heuristic Estimates $h(n)$

Node	$h(n)$
A	8
B	8
C	6
D	5
E	1
F	4
G	0

Important: This is a **directed** graph — edges go one way only (State $\rightarrow$ Next). Note the self-loop $C \rightarrow C$ with cost 0 — this is a degenerate edge and should be ignored during search (it does not change the state or advance toward the goal). Also note the edge $D \rightarrow C(2)$: returning from D to C is allowed since the graph is directed — C is reachable from both A and D. No reverse edges from $B \rightarrow A$, $C \rightarrow A$, $D \rightarrow B$, etc. exist unless listed in the table above.

Part a — Problem Formulations [2 marks]

Question: Give the initial state, goal test, operators, and path cost for each problem.

(i) Travelling Salesman Problem (TSP)

- State:** A pair (ℓ, V) where ℓ is the current city and $V \subseteq \{\text{all } N \text{ cities}\}$ is the set of *unvisited* cities. *Example midway:* (Dhaka, {Chittagong, Sylhet}) means we are in Dhaka and still need to visit Chittagong and Sylhet.
- Initial state:** (start city, {all $N-1$ other cities}).
- Goal test:** $\ell = \text{start city}$ **and** $V = \emptyset$ (returned to start after visiting all cities exactly once).
- Operators:** Travel($\text{city}_i \rightarrow \text{city}_j$) where $\text{city}_j \notin V$ or ($V = \emptyset$ and $\text{city}_j = \text{start}$). Each move removes city_j from V .
- Path cost:** Sum of distances (or costs) of all travelled roads. Goal: find the tour with **minimum total cost**.

Complexity note: With N cities there are $(N-1)!/2$ distinct tours — factorial growth. TSP is NP-hard, so exhaustive search is infeasible for $N > 20$. Heuristic search (A^*) with an admissible heuristic (e.g. minimum spanning tree of remaining cities) can find optimal or near-optimal tours for moderate N .

(ii) Missionaries & Cannibals

State: A triple (M_L, C_L, B) where M_L = missionaries on left bank (0–3), C_L = cannibals on left bank (0–3), $B \in \{L, R\}$ = boat position. The right bank has $3-M_L$ missionaries and $3-C_L$ cannibals.

Initial state: $(3, 3, L)$ — all six people and the boat on the left bank.

Goal test: $(0, 0, R)$ — all six people and the boat on the right bank.

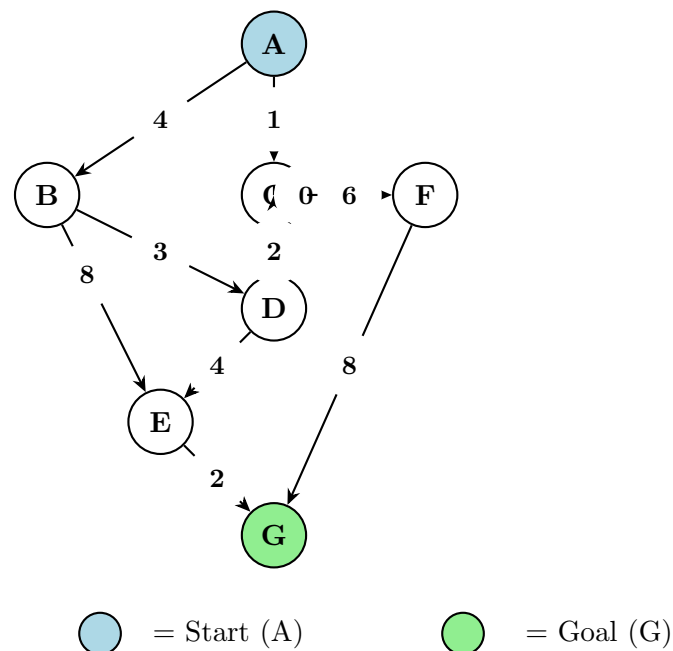
Operators: Move k missionaries and j cannibals across the river **by boat**, where $1 \leq k+j \leq 2$ (boat capacity 2), and the resulting state must be **legal**: on both banks, $M \geq C$ or $M = 0$ (missionaries never outnumbered by cannibals on either side).

Possible boat loads: $(1M, 0C)$, $(2M, 0C)$, $(0M, 1C)$, $(0M, 2C)$, $(1M, 1C)$.

Path cost: Number of boat trips (1 per crossing). Goal: find a sequence with **minimum trips**.

State space size: Without constraints: $4 \times 4 \times 2 = 32$ states. After legality filtering, only about 16–20 are valid. The problem has a known shortest solution of **11 crossings**.

Part b(i) — State-Space Diagram [0.5 mark]



Notes on the graph structure:

- $C \rightarrow C(0)$ is a zero-cost self-loop — degenerate; ignored in search.
- $D \rightarrow C(2)$ creates a directed 2-cycle $C \rightleftharpoons D$ ($C \rightarrow D$ costs 2, $D \rightarrow C$ costs 2).
- B has two outgoing edges: $B \rightarrow D(3)$ and $B \rightarrow E(8)$. $B \rightarrow E$ is an expensive direct path.
- The cheap route to the goal passes through $D \rightarrow E(4) \rightarrow G(2)$.
- From A, the best first step is C (cost 1), not B (cost 4).

Part b(ii) — Search Algorithm Traces [4.5 marks]

Question: Trace UCS, Greedy search, and A* from A (initial) to G (goal). At each step: show the node being expanded and the fringe contents. Ties broken alphabetically.

Algorithm 1 — Uniform-Cost Search (UCS)

Key idea: Expand the node with the **lowest path cost** $g(n)$. Fringe is a min-priority queue sorted by $g(n)$. Once a node is expanded (closed), it is never re-expanded. If a cheaper path to a fringe node is found, update its cost.

Fringe notation: Node[g] sorted ascending by g . **Closed set:** {nodes already expanded}.

Step	Node	g	Fringe after expansion (sorted by g)
1	A	0	C[1], B[4] $A \rightarrow C$: $g=1$. $A \rightarrow B$: $g=4$. Both new.
2	C	1	D[3], B[4], F[7] $C \rightarrow C$: self-loop, ignore. $C \rightarrow D$: $g=3$ new. $C \rightarrow F$: $g=7$ new.
3	D	3	B[4], E[7], F[7] $D \rightarrow C$: $g=5$, C already closed $\Rightarrow$ skip. $D \rightarrow E$: $g=7$ new.
4	B	4	E[7], F[7] $B \rightarrow D$: $g=7$, D already closed $\Rightarrow$ skip. $B \rightarrow E$: $g=12 > E[7]$ in fringe $\Rightarrow$ keep E[7].
5	E	7	F[7], G[9] Tie E[7], F[7]: E before F (alphabetical). $E \rightarrow G$: $g=9$ new.
6	F	7	G[9] $F \rightarrow G$: $g=15 > G[9]$ in fringe $\Rightarrow$ keep G[9].
7	G	9	GOAL FOUND

UCS Solution: $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$ **Total cost:** $1 + 2 + 4 + 2 = 9$
 Nodes expanded (in order): A, C, D, B, E, F, G (7 nodes total)
 UCS guarantees the optimal path because it explores in order of increasing $g(n)$ and step costs are non-negative.

Algorithm 2 — Greedy Best-First Search

Key idea: Expand the node with the **lowest heuristic** $h(n)$. Fringe sorted by h ascending. Path cost $g(n)$ is *completely ignored*. This is fast but not guaranteed optimal.

Fringe notation: Node[h] sorted ascending by h .

Step	Node	h	Fringe after expansion (sorted by h)
1	A	8	C[6], B[8] $A \rightarrow C$: $h=6$. $A \rightarrow B$: $h=8$.
2	C	6	F[4], D[5], B[8] $C \rightarrow F$: $h=4$. $C \rightarrow D$: $h=5$. $C \rightarrow C$ ignored.
3	F	4	G[0], D[5], B[8] $F \rightarrow G$: $h=0$. Goal in fringe with lowest h .
4	G	0	GOAL FOUND

Greedy Solution: $A \xrightarrow{1} C \xrightarrow{6} F \xrightarrow{8} G$ **Total cost:** $1 + 6 + 8 = 15$ (suboptimal!)
 Nodes expanded: A, C, F, G (only 4 nodes — extremely fast)
 Greedy blindly chased the heuristic: from C, F has $h=4$ (lowest), so it went $C \rightarrow F$, but $F \rightarrow G$ costs 8 — the most expensive path to the goal. It completely missed the cheap $C \rightarrow D \rightarrow E \rightarrow G$ route (cost 9) because D's $h=5$ looked slightly worse than F's $h=4$.

Algorithm 3 — A* Search

Key idea: Expand the node with the **lowest** $f(n) = g(n) + h(n)$. Fringe sorted by f ascending. Ties in f broken alphabetically. Closed set prevents re-expansion.

Fringe notation: Node[g, h, f] sorted ascending by f .

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	A	0	8	8	C[1,6, 7], B[4,8, 12] $A \rightarrow C$: $f=1+6=7$. $A \rightarrow B$: $f=4+8=12$.
2	C	1	6	7	D[3,5, 8], F[7,4, 11], B[4,8, 12] $C \rightarrow D$: $g=3, f=8$. $C \rightarrow F$: $g=7, f=11$. $C \rightarrow C$ ignored.
3	D	3	5	8	E[7,1, 8], F[7,4, 11], B[4,8, 12] $D \rightarrow C$: C closed $\Rightarrow$ skip. $D \rightarrow E$: $g=7, f=8$. Tie D($f=8$) and E($f=8$): D already expanded, E is in fringe.
4	E	7	1	8	G[9,0, 9], F[7,4, 11], B[4,8, 12] $E \rightarrow G$: $g=9, f=9$.
5	G	9	0	9	GOAL FOUND

A* Solution: $A \xrightarrow{1} C \xrightarrow{2} D \xrightarrow{4} E \xrightarrow{2} G$ **Total cost:** $1 + 2 + 4 + 2 = 9$ (optimal!)

Nodes expanded: A, C, D, E, G (only 5 nodes — fewer than UCS's 7)

A* found the optimal path and expanded **only 5 nodes** compared to UCS's 7, because the heuristic $h(n)$ guided it away from the expensive B and F branches. Note that **B was never expanded** — its $f=12$ was always worse than the alternatives ahead of it in the fringe.

Algorithm Comparison for 2023 Graph:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
UCS	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$	9	7	Yes
Greedy	$A \rightarrow C \rightarrow F \rightarrow G$	15	4	No
A*	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$	9	5	Yes

Why does A* outperform UCS here? The heuristic $h(n)$ provides a useful lower bound. After expanding A and C, A* sees that D has $f=8$ (promising — $h(D)=5$ is close to the true remaining cost of 6) while B has $f=12$ (unpromising — $h(B)=8$ overestimates its potential). A* expands D next and finds the optimal path without ever touching B. UCS, lacking a heuristic, expands B unnecessarily at step 4 before reaching the goal.

Admissibility check: Is $h(n)$ admissible? Let's verify:

- $h(A)=8 \leq h^*(A)=9$ ✓
- $h(C)=6 \leq h^*(C)=8$ ✓
- $h(D)=5 \leq h^*(D)=6$ ✓
- $h(E)=1 \leq h^*(E)=2$ ✓
- $h(B)=8 \leq h^*(B)=5$? **No!** $h^*(B) = \min(3+4+2, 8+2) = 9$. Wait — $B \rightarrow D \rightarrow E \rightarrow G = 3+4+2 = 9$, so $h(B)=8 < 9$ ✓. And $B \rightarrow E \rightarrow G = 8+2 = 10$. So $h^*(B)=9$, $h(B)=8$ is admissible.

- $h(F)=4 \leq h^*(F)=8$ ✓

Yes, the heuristic is admissible — $h(n) \leq h^*(n)$ for all nodes. This is why A* found the optimal solution.

Part c — IDDFS: Synthesising Benefits of BFS and DFS [2 marks]

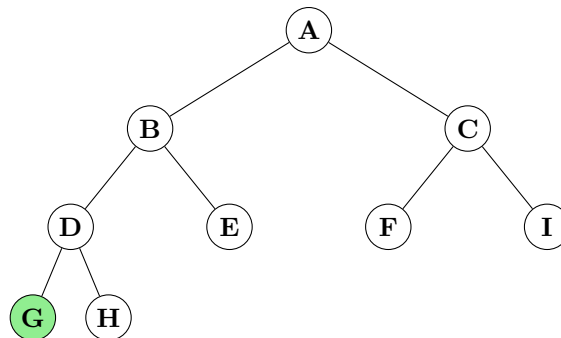
Question: Illustrate with an appropriate example how IDDFS synthesises the benefits of both BFS and DFS — achieving optimality, completeness, and linear space complexity. Nevertheless, time complexity deteriorates in comparison to both BFS and DFS.

The Core Idea

IDDFS (Iterative Deepening Depth-First Search) runs DFS repeatedly with an increasing depth cutoff: first $L=0$, then $L=1$, $L=2$, $\dots$, until the goal is found. At each iteration, the search is a standard depth-limited DFS from the root.

Illustrative Example

Consider the following binary tree where G (shaded) is the shallowest goal at depth 3:



IDDFS Execution:

L	Nodes visited (DFS order)	Outcome
0	A	$A \neq G$. No solution at depth 0.
1	A, B, C	Neither B nor C is G. No solution at depth 1.
2	A, B, D, E, C, F, I	No goal at depth 2.
3	A, B, D, G ← GOAL FOUND	Found! Shallowest goal at depth 3.

How IDDFS Inherits Each Algorithm's Strengths

Property	BFS	DFS	IDDFS
Completeness	✓ Complete (finds solution if one exists)	× Incomplete (may loop forever on infinite branches)	✓ Complete — like Depth-limited DFS and avoids infinite loops because it backtracks at the cutoff
Optimality	✓ Optimal for uniform step costs (finds shallowest solution)	× Not optimal (may find a deep solution first)	✓ Optimal for uniform costs — like BFS. Because it explores all nodes at depth $d-1$ before any at depth d , the first solution found is the shallowest
Space	× $O(b^d)$ — stores entire frontier. For $b=10, d=10$, that's 10 billion nodes.	✓ $O(bm)$ — stores only one path + siblings.	✓ $O(bd)$ — like DFS. Only the current path (depth d) plus $b-1$ siblings at each level are in memory. $b=10, d=10$: ≈ 100 nodes
Time	$O(b^d)$ — visits each node once	$O(b^m)$ — may go very deep, but each node visited once	× $O(b^d)$ — worse constant factor . Nodes near the root are revisited many times. Overhead factor is large (e.g. for $b=10$, about 10 times BFS).

Why Time Complexity Deteriorates

The key weakness of IDDFS is **repeated work**. Consider the root node A: it is expanded in *every* iteration ($L=0, 1, 2, 3$) — 4 times. Node B is expanded at $L=2$ and $L=3$ — twice. Only the deepest nodes (depth L) are expanded just once.

However, in a tree with branching factor b , the **number of nodes at the deepest level dominates**:

$$\begin{aligned}
 \text{Total expansions} &= (d+1) \cdot 1 + d \cdot b + (d-1) \cdot b^2 + \dots + 1 \cdot b^d \\
 &= b^d \left(1 + \frac{1}{b} + \frac{1}{b^2} + \dots \right) \approx b^d \cdot \frac{b}{b-1}
 \end{aligned}$$

For $b = 10$, the overhead is only $\frac{10}{9} \approx 1.11\times$ — a small price for the enormous space savings ($O(bd)$ vs $O(b^d)$). This is why IDDFS is the **preferred uninformed search strategy** when the state space is large and step costs are uniform.

When IDDFS Is NOT Optimal

IDDFS finds the **shallowest** goal (fewest edges), not the lowest-cost goal. With non-uniform step costs, a deeper path with lower total cost may exist. In such cases, UCS or A* is required. For the 2024 exam graph, IDDFS found $A \rightarrow B \rightarrow E \rightarrow G$ (cost 16, 3 hops) while UCS found $A \rightarrow C \rightarrow H \rightarrow G$ (cost 13, 3 hops) — both at the same depth, but UCS found the cheaper one because it considers edge costs, not just depth.

Exam summary — IDDFS:

- **Completeness:** Inherited from BFS — guaranteed to find a solution if one exists.
- **Optimality:** Inherited from BFS — optimal for *uniform* step costs (finds shallowest solution).
- **Space:** Inherited from DFS — $O(bd)$ linear space, practical for large search spaces.
- **Time:** Worse than both BFS and DFS individually — nodes near the root are re-expanded each iteration. However, the overhead is only a constant factor ($\approx \frac{b}{b-1}$), acceptable given the space savings.

Examination 2022

Note: Search questions in 2022 are **scattered across two sections**:

- **Section A, Q.A-4(a):** DFS vs BFS theory (2 marks)
- **Section B, Q.B-4(b):** Graph trace — UCS, Greedy, A* (3 marks) — *identical search space to 2023*

Section A — Q.A-4(a): DFS vs BFS [2 marks]

Question: What is an AI Technique? Write down the advantages and disadvantages of Depth-First Search and Breadth-First Search.

AI Technique

An **AI Technique** is a method that exploits knowledge, represented in a form that:

1. **Captures generalisations** — groups situations sharing important properties together, avoiding separate representations for every individual case.
2. **Can be understood** by the people providing it — knowledge must be inspectable and modifiable.
3. **Can be easily modified** to correct errors and reflect new information.
4. **Can be used in many situations** even if not perfectly complete or accurate.
5. **Reduces the search space** by narrowing the range of possibilities that must be considered.

In short: an AI technique is a **knowledge-driven method for controlling search** so that solutions are found efficiently without exhaustive enumeration of all possibilities. DFS and BFS are two fundamental *uninformed* (blind) search techniques — they use no domain-specific knowledge, only the structure of the search tree.

Depth-First Search (DFS)

Advantages	Disadvantages
1. Low memory: Stores only the current path (b siblings per level), giving $O(bm)$ space. For deep trees, this is the decisive advantage.	1. Not complete: Fails on infinite-depth trees. DFS may follow an infinite branch forever and never find a solution elsewhere.
2. Simple implementation: Easy to code recursively or with a stack (LIFO).	2. Not optimal: The first solution found may be very deep in the tree. DFS has no mechanism to prefer shorter paths.
3. Fast to a solution if lucky: If the goal is deep and branching is low, DFS may find it with very few node expansions.	3. Susceptible to loops: Without a closed/visited set, DFS can cycle infinitely.
4. Backtracking-friendly: Natural fit for constraint satisfaction and puzzle-solving where partial solutions are built and abandoned.	4. May miss shallow goals: DFS goes deep first — a goal at depth 2 may be found only after exhausting paths to depth 50.

Breadth-First Search (BFS)

Advantages	Disadvantages
1. Complete: Guaranteed to find a solution if one exists (provided b is finite).	1. Exponential memory: Must store the entire frontier — $O(b^d)$ space. For $b=10$, $d=12$: 1 trillion nodes in memory. This makes BFS infeasible for most real problems.
2. Optimal (uniform cost): Finds the shallowest solution. If step costs are all equal, this is the optimal cost.	2. Exponential time: $O(b^d)$ — same as the space requirement. Practical only for shallow trees.
3. Systematic: Processes the entire tree level by level — no path is ever skipped or overlooked.	3. Slow to find deep goals: Must explore all nodes at depth $d-1$ before checking any at depth d . If the goal is at depth 10, all 9 shallow levels must be fully expanded first.
4. Good for shallow goals: If the goal is near the root, BFS finds it quickly.	4. Queue implementation overhead: Maintaining the frontier as a FIFO queue with $O(1)$ enqueue/dequeue while also tracking visited states adds complexity.

When to Use Which

Criterion	Use DFS	Use BFS
Memory is limited	Yes — $O(bm)$ space	No — $O(b^d)$ space
Goal is known to be deep	Yes — goes deep fast	No — explores all shallow levels first
Goal is known to be shallow	No — may waste time deep	Yes — finds it early
Need shortest path	No — not optimal	Yes (uniform costs)
Infinite / very deep tree	No — may not terminate	Yes — complete

The IDDFS resolution: Iterative Deepening DFS combines the space advantage of DFS ($O(bd)$ memory) with the completeness and optimality of BFS. For most practical uninformed search problems where memory is the bottleneck, **IDDFS is the preferred choice**. It was tested in 2024 Q2, 2023 Q2(c), 2021 Q3(b), and 2020

Q3(b) — a core exam pattern.

Section B — Q.B-4(b): Graph Trace [3 marks]

Question: Suppose you have the following search space (State, Next, Cost as given in Table 1, $h(n)$ as given in Table 2). Assume initial state A and goal state G. Show how each of the following search strategies would create a search tree to find a path from A to G: **Uniform cost, Greedy search, and A***. At each step show which node is being expanded and the content of fringe.

The **2022 search space is identical to the 2023 search space** (same states, same edges, same $h(n)$ values). The full step-by-step traces for UCS, Greedy, and A* on this graph are given in **Section 2.3** (Examination 2023, Part b(ii)). Below is the summary.

Algorithm	Path	Cost	Nodes Expanded
UCS	A → C → D → E → G	9	7 (A, C, D, B, E, F, G)
Greedy	A → C → F → G	15	4 (A, C, F, G)
A*	A → C → D → E → G	9	5 (A, C, D, E, G)

Key observation — same graph appears 3 times: The identical search space (A→B(4), A→C(1), B→D(3), B→E(8), C→D(2), C→F(6), D→C(2), D→E(4), E→G(2), F→G(8)) with the same $h(n)$ values is tested in **2020 Q2, 2022 B-4(b), and 2023 Q2**. Mastering the traces on this one graph gives you answers for 3 separate exam years. The cheapest path is always A→C→D→E→G costing 9.

Examination 2021

Note: Search questions in 2021 appear across two Q-numbers:

- **Q2 (9.75 marks):** Full search question — problem formulation + graph traces (DFID, Greedy, A*)
- **Q3(b) (1.75 marks):** DFID vs IDA* — comparative theory

Q2(a) — State Space Search Problem [0.75 mark]

Question: What is the state space search problem?

A **state space search problem** is a formal representation of a problem as a directed graph where:

- **States** are vertices — each represents a possible configuration of the world.
- **Actions** (or operators) are directed edges — transitions from one state to another.
- A **solution** is a sequence of actions (a path) from an initial state to a goal state.
- The search algorithm **systematically explores** the state space to find this path.

Formally, a state space search problem is defined by four components:

1. **Initial state** — the starting configuration.
2. **Goal test** — a function that determines whether a given state is a goal.
3. **Successor function** (operators) — given a state, returns all (action, next_state) pairs.
4. **Path cost function** — assigns a numeric cost to each path (typically additive).

The **working principle** of a basic search algorithm:

1. Initialise the fringe with the start state.
2. Loop: remove a state from the fringe; if it satisfies the goal test, return the solution path; otherwise, expand it (generate successors) and add them to the fringe.
3. Repeat until the fringe is empty (failure) or goal is found (success).

Q2(b) — Problem Formulations [3 marks]

Question: Give the initial state, goal test, successor function, cost function, and working principle of a basic search algorithm for each problem.

(i) Map Coloring Problem

Problem: Using only 4 colors, color a planar map such that no two adjacent regions share the same color.

State: A partial assignment of colors $\{R, G, B, Y\}$ to a subset of regions $\{r_1, r_2, \dots, r_N\}$. Unassigned regions are marked as $\emptyset$. *Example:* $\{r_1=R, r_2=G, r_3=\emptyset, \dots\}$.

- Initial state:** All N regions unassigned: $\{r_i = \emptyset \mid i = 1, \dots, N\}$.
- Goal test:** All regions are assigned a color, and for every pair of adjacent regions (r_i, r_j) , $color(r_i) \neq color(r_j)$.
- Successor function:** Pick an unassigned region r_i and assign it any of the 4 colors $\{R, G, B, Y\}$ that is **not** already used by any of r_i 's adjacent (already-colored) regions. If no color is valid, backtrack.
- Path cost:** Typically 1 per assignment (uniform cost). Since all solutions assign exactly N colors, any valid solution has cost N . The goal is to find *any* valid coloring, not necessarily optimal.
- Working principle:** This is a **constraint satisfaction problem (CSP)**. Backtracking DFS is the natural fit: assign a color to the next variable, check constraint consistency, backtrack on conflict. DFS is preferred over BFS here because all solutions are at the same depth N , and DFS's low memory allows backtracking through a deep search tree.

(ii) Monkey & Bananas Problem

Problem: A 3-ft monkey is in a room with an 8-ft ceiling. Bananas hang from the ceiling. Two 3-ft crates are available (stackable, movable, climbable). The monkey must get the bananas.

- State:** A 4-tuple $(monkey_pos, crate1_pos, crate2_pos, has_bananas)$ where positions are discrete locations in the room and $has_bananas \in \{\text{true}, \text{false}\}$. If the monkey is on a crate, its effective height increases by the crate height (3 ft per crate; stacking 2 crates gives 6 ft height). The monkey can reach the bananas only if total height ≥ 8 ft.
- Initial state:** $(floor_position, initial_crate1_pos, initial_crate2_pos, false)$ — monkey on floor, crates at their initial locations, no bananas.
- Goal test:** $has_bananas = \text{true}$.
- Successor function (operators):**
1. **Walk**($x \rightarrow y$): Monkey moves from position x to y on the floor (not on a crate).
 2. **Push**(crate, $x \rightarrow y$): Monkey pushes a crate from x to y (must be on floor, adjacent to crate).
 3. **Climb**(crate): Monkey climbs onto a crate (must be adjacent to it on the floor).
 4. **ClimbDown**(): Monkey climbs down from crate to floor.
 5. **Stack**(crate1, crate2): Place crate1 on top of crate2 (monkey must be on floor, crates adjacent).
 6. **Grasp**(): Monkey grabs bananas (must be within reach — total height ≥ 8 ft).
- Path cost:** Number of actions (1 per action). Goal: find the **shortest** sequence to get the bananas.

Key insight: The monkey needs height ≥ 8 ft. One crate gives $3+3=6$ ft (not enough). Two crates stacked give $3+3+3=9$ ft (enough). So the solution requires: walk to crate1, push crate1 under bananas, walk to crate2, push crate2 to same spot, stack crate2 on crate1, climb, grasp.

Q2(c) — Graph Traces [5 marks]

Problem Data

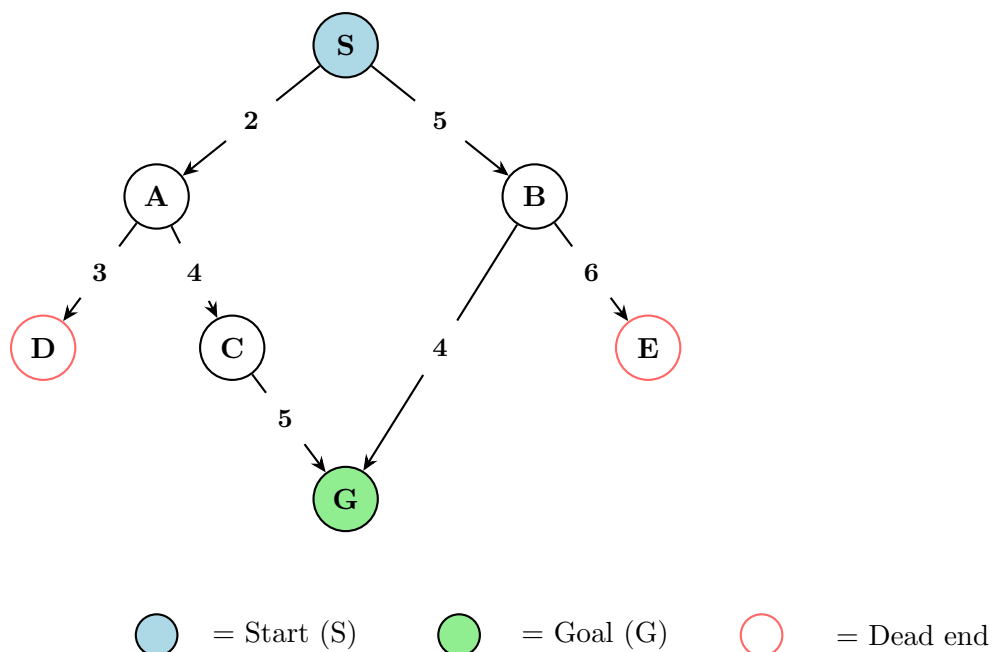
Table 1: Directed Edges and Costs

From	To	Cost
S	A	2
S	B	5
A	C	4
A	D	3
B	E	6
C	G	5
B	G	4

Table 2: Heuristic Estimates $h(n)$

Node	$h(n)$
S	9
A	9
B	4
C	5
D	∞
E	∞
G	0

Graph reconstruction: The exact edge costs from the 2021 past paper Figure 1 are reconstructed from the given $h(n)$ values. The heuristic is **exact** (admissible and consistent): $h(n) = h^*(n)$ for every node. D and E are dead ends ($h=\infty$ means no path to G exists from these nodes).



Question: Assume start=S, goal=G. Show search trees for **Depth First Iterative Deepening (DFID)**, **Greedy**, and **A*** — at each step show node expanded and the fringe. Also report the eventual algorithm and solution cost.

Algorithm 1 — Depth-First Iterative Deepening (DFID)

Key idea: Perform depth-limited DFS with increasing cutoff $L = 0, 1, 2, \dots$ until goal is found. The fringe is a **stack** (LIFO). At each iteration, restart from S. Nodes are pushed in **reverse alphabetical** order so they are popped in alphabetical order.

Iteration $L = 0$:

Step	Action	Stack
1	Pop S($d=0$). $S \neq G$. At depth limit.	[]

No solution at $L = 0$.

Iteration $L = 1$:

Step	Action	Stack
1	Pop S($d=0$). Expand: push B, then A (A on top).	A(1), B(1)
2	Pop A($d=1$). $A \neq G$. At limit.	B(1)
3	Pop B($d=1$). $B \neq G$. At limit.	[]

No solution at $L = 1$.

Iteration $L = 2$:

Step	Action	Stack
1	Pop S($d=0$). Push B, A (A on top).	A(1), B(1)
2	Pop A($d=1$). Expand (excl. S): C, D. Push D, C (C on top).	C(2), D(2), B(1)
3	Pop C($d=2$). Expand (excl. A): G. Push G. But C is at depth $d=2$, so G would be at depth $3 > L=2$. Cannot push! C at limit. $C \neq G$.	D(2), B(1)
4	Pop D($d=2$). D at limit.	B(1)
5	Pop B($d=1$). Expand (excl. S): E, G. Push G, E (E on top).	E(2), G(2)
6	Pop E($d=2$). E at limit.	G(2)
7	Pop G($d=2$). G = GOAL!	GOAL FOUND

DFID Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Depth: 2** **Cost:** $5 + 4 = 9$

DFID finds the **shallowest** goal (depth 2, via $S \rightarrow B \rightarrow G$) at iteration $L = 2$. DFID is complete and optimal for uniform step costs, but here with non-uniform edge costs, it finds the shallowest goal rather than necessarily the cheapest — however, in this graph, $S \rightarrow B \rightarrow G$ happens to also be the optimal-cost path.

Algorithm 2 — Greedy Best-First Search

Fringe notation: Node[h] sorted ascending by h .

Step	Node	h	Fringe after expansion (sorted by h)
1	S	9	B[4], A[9] $S \rightarrow B$: $h=4$. $S \rightarrow A$: $h=9$. B has lowest h , promising.
2	B	4	G[0], E[∞], A[9] $B \rightarrow G$: $h=0$ (goal!). $B \rightarrow E$: $h=\infty$ (dead end, goes to back).
3	G	0	GOAL FOUND

Greedy Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Cost:** $5 + 4 = 9$ (optimal — by luck!)

Nodes expanded: S, B, G (only 3 nodes)

Greedy found the optimal path because the heuristic h strongly prefers B ($h=4$) over A ($h=9$), and B directly connects to G. Greedy never even explored the A-branch, saving work.

Algorithm 3 — A* Search

Fringe notation: Node[g, h, f] sorted ascending by f .

Step	Node	g	h	f	Fringe after expansion (sorted by f)
1	S	0	9	9	B[5,4,9], A[2,9,11] $S \rightarrow B$: $f=5+4=9$. $S \rightarrow A$: $f=2+9=11$.
2	B	5	4	9	G[9,0,9], A[2,9,11], E[11, ∞ , ∞] $B \rightarrow G$: $f=9$. $B \rightarrow E$: $f=\infty$, deprioritised. Tie G[9],B[9]: B already expanded, G next.
3	G	9	0	9	GOAL FOUND

A* Solution: $S \xrightarrow{5} B \xrightarrow{4} G$ **Cost:** $5 + 4 = 9$ (optimal!)

Nodes expanded: S, B, G (only 3 nodes — same as Greedy!)

A* mirrors Greedy in this graph because the heuristic is **exact** ($h(n) = h^*(n)$ for all n), meaning $f(n) = g(n) + h^*(n)$ — the exact total cost to the goal through n . When the heuristic is exact, A* expands only nodes on optimal paths. A* never expanded A ($f=11 > 9$) — it was correctly identified as lying on a suboptimal path.

Algorithm Comparison for 2021 Graph:

Algorithm	Path Found	Cost	Nodes Expanded	Optimal?
DFID	$S \rightarrow B \rightarrow G$	9	7 (over all iterations)	Yes (this graph)
Greedy	$S \rightarrow B \rightarrow G$	9	3	Yes (by luck)
A*	$S \rightarrow B \rightarrow G$	9	3	Yes

Why Greedy found the optimal path here: The heuristic is **exact** — $h(n)$ equals the true optimal cost to G from every node. Since $h(B)=4$ ($B \rightarrow G$ costs 4) and $h(A)=9$ ($A \rightarrow C \rightarrow G$ costs 9), Greedy correctly identifies B as the better first step. However, **this is not guaranteed in general**. With a less accurate heuristic, Greedy could be misled. The 2023 graph demonstrated exactly this: $h(F)=4$ looked slightly better than $h(D)=5$, but $F \rightarrow G$ cost 8 while $D \rightarrow E \rightarrow G$ cost 6, making the D-path cheaper overall.

Q3(b) — DFID vs IDA* [1.75 marks]

Question: Differentiate between DFID and IDA* algorithms. Demonstrate how DFID incorporates the advantages of BFS and DFS with an example.

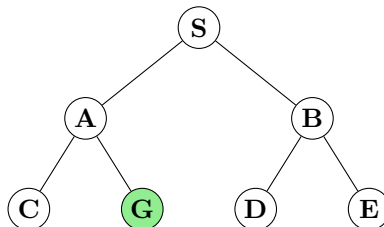
DFID (Depth-First Iterative Deepening)

DFID runs depth-limited DFS with cutoff $L = 0, 1, 2, \dots$ until the goal is found. The cutoff is purely **structural** — it limits the number of edges (depth) from the root.

How DFID combines BFS and DFS advantages:

Property	BFS	DFS	DFID (inherits)
Completeness	✓ Complete	× May loop forever	✓ Complete — BFS. Each iteration bounded depth; branches cannot t
Optimality	✓ Optimal (uniform cost)	× Not optimal	✓ Optimal for uniform costs — BFS. Finds shortest solution first.
Space	× $O(b^d)$	✓ $O(bm)$	✓ $O(bd)$ — from BFS. Only stores one p

Example — Binary Tree (branching factor $b = 2$):



DFID execution on this tree:

- $L=0$: Visit S. Not G.
- $L=1$: Visit S, A, B. Not G.
- $L=2$: Visit S, A, C, **G** ← Found at depth 2.

BFS would: Store all nodes at depth 2 (4 nodes) in the frontier — $O(b^d)$ memory. **DFS would:** Store only the path $S \rightarrow A \rightarrow C$ (3 nodes) — $O(bd)$ memory. **DFID:** Also stores only the path $S \rightarrow A \rightarrow C$ (3 nodes) — $O(bd)$ memory, but with the completeness guarantee of BFS.

IDA* (Iterative Deepening A*)

IDA* is the **informed** counterpart of DFID. Instead of a depth cutoff, it uses an **f -cost cutoff**:

- Each iteration performs DFS, pruning any node where $f(n) = g(n) + h(n) > cutoff$.
- The initial cutoff is $f(start) = h(start)$.
- After each iteration, the cutoff is increased to the **smallest f -value that exceeded the previous cutoff**.
- The algorithm terminates when the goal is found (optimal) or no nodes remain below ∞ cutoff.

Key Differences

Criterion	DFID	IDA*
Cutoff type	Depth (structural — number of edges from root)	f-cost = $g(n) + h(n)$ (informed uses heuristic knowledge)
Search type	Uninformed (blind)	Informed (uses heuristic h)
Optimality guarantee	Optimal for <i>uniform</i> edge costs only	Optimal for <i>any</i> edge costs, provided h is admissible
Overhead per iteration	Lower — just checks depth $\leq L$	Higher — must compute $f(n) = g(n) + h(n)$ for every node visited
When to use	Unknown/unreliable heuristic, uniform costs	Good admissible heuristic available, non-uniform costs
Space complexity	$O(bd)$ — stores current path	$O(bd)$ — stores current path (same as DFID)
Cutoff progression	$L = 0, 1, 2, \dots$ (linear, fixed step)	$f_{cutoff} = \min\{f(n) \mid n \text{ pruned in last iteration}\}$ (adaptive, data-driven)

Exam summary:

- **DFID** = IDDFS = depth-bounded iterative DFS. Uninformed. Uses depth as cutoff.
- **IDA*** = f -cost-bounded iterative DFS. Informed. Uses $f(n) = g(n) + h(n)$ as cutoff.
- Both algorithms achieve $O(bd)$ space — their key advantage over BFS and A* (which both require $O(b^d)$ space).
- DFID is to BFS what IDA* is to A*: the iterative-deepening, space-efficient version.
- In practice, IDA* is preferred for problems with good heuristics and large state spaces (e.g., the 15-puzzle).

Examination 2020

Note: 2020 has the broadest coverage of search questions — spanning Q2, Q3(b), Q3(c) in Section A and Q7(b) in Section B.

- **Q2 (8.75 marks):** Problem formulation + search space formulation + graph trace (UCS, Greedy, A*)
- **Q3(b) (2 marks):** IDDFS — how it combines BFS + DFS benefits
- **Q3(c) (1.75 marks):** BFS admissibility
- **Q7(b) (2 marks):** A* heuristic — consequences of under/overestimation

Q2(a) — Formulating a Search Problem [1.75 marks]

Question: How can you formulate a search problem? Give examples of data structures that can be used to represent a state-space both in explicit and implicit ways.

Formulating a Search Problem

A search problem is formulated by defining **four components**:

1. **Initial state** — the state in which the agent starts. *Example:* In a route-finding problem, the starting city.
2. **Goal test** — a function $\text{isGoal}(s) \rightarrow \{\text{true}, \text{false}\}$ that determines whether a given state s satisfies the goal condition. Can be an explicit set of goal states or an abstract condition.
3. **Successor function** $\text{Succ}(s) \rightarrow \{(a, s')\}$ — given a state s , returns all (action, resulting-state) pairs reachable by legal actions.
4. **Path cost function** $g(p)$ — assigns a numeric cost to a path p (typically the sum of step costs along the path).

Together, these four components define the **search space** — a directed graph where vertices are states and edges are actions. A **solution** is a path from the initial state to any state satisfying the goal test. An **optimal solution** has the lowest path cost among all solutions.

Representing State-Spaces

1. Explicit Representation (State-Space Graph):

The entire state-space graph is stored in memory before search begins. Each state and its connections are enumerated.

Data structures used:

- **Adjacency list:** For each state, store a list of (neighbor, cost) pairs. Memory: $O(|V| + |E|)$.
- **Adjacency matrix:** A $|V| \times |V|$ matrix where entry $[i][j]$ holds the cost of the edge from state i to state j (or ∞ if no edge). Memory: $O(|V|^2)$.
- **Edge list:** Store all edges as triples (from_state, to_state, cost). Memory: $O(|E|)$.

Limitation: Only practical for **small** state spaces (e.g., the Romania map with 20 cities). For problems like the 8-puzzle ($9!/2 = 181,440$ states) or chess ($\sim 10^{47}$ states), explicit storage is infeasible.

2. Implicit Representation (Successor Function):

The state space is **generated on demand** during search. Only the current state and the successor function are stored; the full graph is never materialised.

Data structures used:

- **State representation:** A compact data structure encoding the current configuration. *Examples:* a tuple (x, y) for grid positions; a 2D array for the 8-puzzle board; a set of boolean variables for satisfiability problems.
- **Successor function (as code):** A function that, given a state, **computes** legal next states algorithmically — no lookup table needed. *Example:* For the Water Jug problem, successors of (x, y) are computed by the 6 pouring/filling rules applied to the current values of x and y .
- **Fringe (frontier):** A queue (FIFO for BFS), stack (LIFO for DFS), or priority queue (for UCS/A*) holding states discovered but not yet expanded. Memory depends on the algorithm, not the state-space size.
- **Closed/explored set:** A hash table or set storing states already expanded, preventing redundant work. Uses $O(|V_{\text{expanded}}|)$ memory.

Advantage: Scales to **enormous** state spaces. The 8-puzzle with 10^5 states is searched without ever storing the full graph. Chess engines use implicit representation — they generate moves on the fly and search to depth 20+ without enumerating all 10^{47} positions.

Q2(b) — Problem Formulations [2 marks]

Question: Give the initial state, goal test, successor function, and cost function for each problem.

(i) Travelling Salesman Problem (TSP)

State: (ℓ, V) where ℓ is the current city and V is the set of unvisited cities.

Initial state: $(\text{start_city}, \{\text{all other } N-1 \text{ cities}\})$.

Goal test: $\ell = \text{start_city}$ **and** $V = \emptyset$.

Successor function: $\text{Travel}(c_i \rightarrow c_j)$ for any $c_j \in V$ (or $c_j = \text{start_city}$ if $V = \emptyset$). Removes c_j from V .

Path cost: Sum of road distances travelled. Goal: find the **minimum-cost** Hamiltonian cycle.

Complexity: $(N-1)!/2$ distinct tours — exponential. This is the canonical NP-hard problem.

(ii) Missionaries & Cannibals

State: (M_L, C_L, B) where $M_L, C_L \in \{0, 1, 2, 3\}$ are missionaries and cannibals on the left bank, and $B \in \{L, R\}$ is the boat position.

Initial state: $(3, 3, L)$.

Goal test: $(0, 0, R)$.

Successor function: Move 1 or 2 people across the river by boat. Valid combinations: $(1M, 0C)$, $(2M, 0C)$, $(0M, 1C)$, $(0M, 2C)$, $(1M, 1C)$. After each move, verify the **safety constraint**: on both banks, $M \geq C$ or $M = 0$.

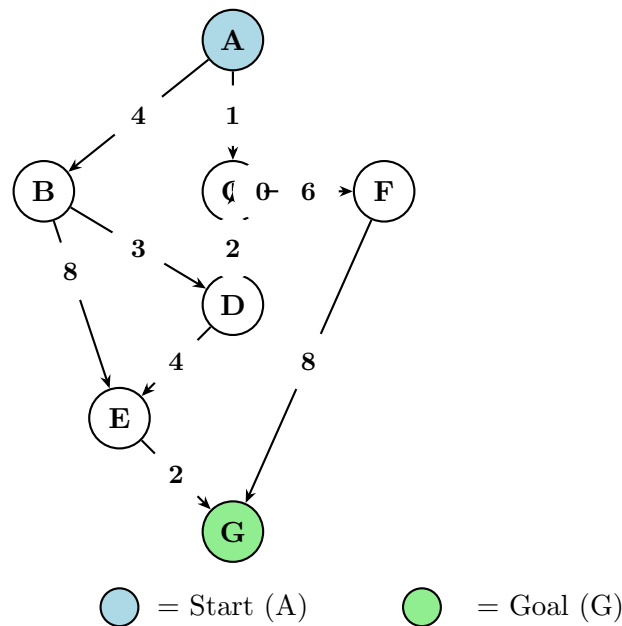
Path cost: 1 per boat crossing. Goal: **minimum crossings** (known to be 11).

Q2(c) — Graph Traces [5 marks]

Question: Using the search space given (see Table), draw the state-space, then trace UCS, Greedy, and A* from A to G. At each step show which node is being expanded and the content of the fringe.

The 2020 search space is identical to the 2022 and 2023 search space (same 6 nodes A-F plus goal G, same directed edges, same $h(n)$ values). The complete step-by-step traces for all three algorithms on this graph are given in **Section 2.3** (Examination 2023, Part b(ii)). Below is the summary and the state-space diagram.

Edge Table			Heuristic $h(n)$	
State	Next	Cost	Node	$h(n)$
A	B	4	A	8
A	C	1	B	8
B	D	3	C	6
B	E	8	D	5
C	C	0	E	1
C	D	2	F	4
C	F	6	G	0
D	C	2		
D	E	4		
E	G	2		
F	G	8		



Algorithm Results Summary:

Algorithm	Path	Cost	Expansions	Optimal?
UCS	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$	9	7	Yes
Greedy	$A \rightarrow C \rightarrow F \rightarrow G$	15	4	No
A*	$A \rightarrow C \rightarrow D \rightarrow E \rightarrow G$	9	5	Yes

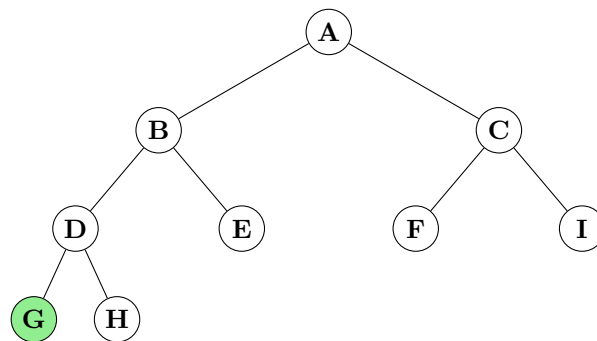
For the full step-by-step traces (node expanded, fringe contents at each step, discussion of why Greedy fails and A* succeeds), see **Section 2.3** (Examination 2023). The traces on this graph are among the most important in the syllabus — the same search space appears in 3 exam years (2020, 2022, 2023).

Q3(b) — IDDFS Combines BFS + DFS [2 marks]

Question: Iterative-deepening (ID) is a type of search algorithm said to combine the benefits of breadth-first and depth-first search. Explain, using a diagram or otherwise, how ID achieves these benefits.

The Problem: BFS guarantees the shortest path but needs $O(b^d)$ memory — infeasible for large search spaces. DFS uses only $O(bm)$ memory but may never terminate on infinite branches and finds suboptimal deep solutions.

The IDDFS Solution: Run depth-limited DFS repeatedly with cutoff $L = 0, 1, 2, \dots$ until the goal is found.



L	Nodes visited (DFS order)	Space used
0	A	1 node (just A)
1	A, B, C	2 nodes (A→B, then A→C)
2	A, B, D, E, C, F, I	3 nodes (A→B→D at deepest)
3	A, B, D, G ← found!	3 nodes

How IDDFS inherits from each:

Property	Inherited from BFS	Inherited from DFS
Completeness	✓ BFS explores level by level, never trapped by infinite branches. IDDFS does the same — depth limits prevent infinite descent.	—
Optimality	✓ BFS finds the shallowest goal. IDDFS does the same — because it exhausts depth $d-1$ before checking depth d , the first goal found is shallowest.	—
Space	—	✓ DFS uses $O(bm)$ memory — only the current path. IDDFS replicates this: at any moment, only $O(bL)$ nodes are stored (the current path plus siblings). For $b=10, L=10$: ~ 100 nodes vs BFS's 10 billion.
Simple implementation	—	✓ Like DFS, IDDFS uses a simple stack or recursion. The outer loop adds minimal complexity.

Cost of the compromise — Time Overhead: IDDFS re-expands nodes near the root: node A is expanded $d+1$ times, depth-1 nodes are expanded d times, etc. However, the deepest level dominates the node count. The overhead factor is $\frac{b}{b-1}$:

$$\text{Total work} = b^d \left(1 + \frac{1}{b} + \frac{1}{b^2} + \dots \right) \approx b^d \cdot \frac{b}{b-1}$$

For $b = 10$, the overhead is only $\frac{10}{9} \approx 1.11\times$ — an acceptable price for reducing memory from $O(b^d)$ to $O(bd)$.

IDDFS = BFS completeness + BFS optimality + DFS space efficiency. The only trade-off is a modest constant-factor increase in time ($\approx 11\%$ for $b=10$), making it the **preferred uninformed search strategy** for large state spaces with uniform costs.

Q3(c) — BFS Admissibility [1.75 marks]

Question: When is breadth-first search an admissible search strategy? Briefly explain.

What “Admissible” Means

In search, an algorithm is **admissible** if it **always finds the optimal (lowest-cost) solution** when one exists. The term is most commonly applied to heuristics ($h(n) \leq h^*(n)$), but it can also describe complete search algorithms.

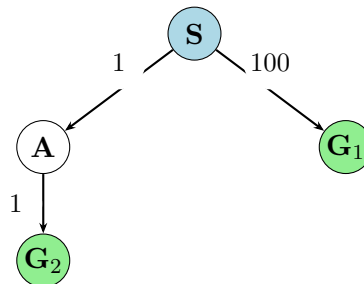
When BFS is Admissible

BFS is admissible **when all step costs are equal (uniform)**. Under this condition:

1. BFS explores the state space **level by level**: all nodes at depth 0, then depth 1, then depth 2, ...
2. Since every step costs the same amount (say, 1), the **depth of a node equals its path cost**: $g(n) = \text{depth}(n)$.
3. Therefore, the **first goal node encountered** is at the shallowest depth, which corresponds to the **minimum path cost**.
4. BFS returns this shallowest goal — which is optimal.

When BFS is NOT Admissible

BFS is **not admissible when step costs are non-uniform**. Consider:



BFS would find G_1 first (depth 1, cost 100) and return it as the solution. But the optimal path is $S \rightarrow A \rightarrow G_2$ (depth 2, cost $1 + 1 = 2$). BFS returns the **shallowest** goal, not the **cheapest**. The solution is suboptimal, so BFS is not admissible when edge costs differ.

Formal Statement

BFS is admissible iff all step costs are equal.

When costs are uniform, depth $\propto$ cost, and the shallowest goal is the cheapest. When costs vary, BFS returns the shallowest goal regardless of cost — this may be suboptimal. For non-uniform costs, use **UCS** (Uniform-Cost Search) instead — it expands by $g(n)$ rather than depth, guaranteeing optimality for any non-negative edge costs.

Q7(b) — A* Heuristic: Underestimate vs Overestimate [2 marks]

Question: What will happen if h' underestimates h in the A* algorithm? What will happen if h' overestimates h ?

Notation clarification: In standard A* notation, $h(n)$ is the heuristic estimate of the cost from n to the goal, and $h^*(n)$ is the *true* optimal cost. The question uses h' for the heuristic and h for the true cost. We adopt the standard convention below.

Case 1: Heuristic UNDERESTIMATES ($h(n) \leq h^*(n)$ — Admissible)

When the heuristic never overestimates the true cost to the goal (it is **admissible**):

Consequence	Explanation
A* is optimal — it will find the lowest-cost solution.	The proof: A* expands nodes in order of $f(n) = g(n) + h(n)$. Since $h(n) \leq h^*(n)$, we have $f(n) \leq g(n) + h^*(n) = \text{true total cost through } n$. The first time A* expands a goal node G , $f(G) = g(G) + 0 = g(G)$. For any unexpanded node n on a cheaper path to a different goal, $f(n) \leq \text{true cost of that cheaper path} < g(G)$. So A* would have expanded n before G — contradiction. Therefore the first expanded goal must be optimal.
May expand more nodes than with a perfect heuristic.	A severe underestimate (e.g., $h(n) = 0$ for all n , which reduces A* to UCS) leads to more node expansions. The closer h is to h^* , the fewer nodes A* expands.
Still complete — will find a solution if one exists.	Admissibility does not affect completeness. A* with any heuristic eventually explores the entire (finite) search space if needed.

Example — admissible (underestimate): $h(\text{Arad}) = 366 \leq h^*(\text{Arad}) = 418$. A* finds the optimal path Arad $\rightarrow$ Sibiu $\rightarrow$ Rimnicu $\rightarrow$ Pitesti $\rightarrow$ Bucharest (cost 418).

Case 2: Heuristic OVERESTIMATES ($h(n) > h^*(n)$ for some n)

When the heuristic **overestimates** the true cost for at least one node (it is **not admissible**):

Consequence	Explanation
A* may NOT be optimal — it can return a suboptimal solution.	If $h(n)$ overestimates the remaining cost through node n , then $f(n)$ is inflated. A* may postpone expanding n in favor of nodes whose f -values are lower but lie on suboptimal paths. A suboptimal goal may be expanded and returned before the optimal path is discovered.
The optimal path may be “hidden.”	A* considers nodes in order of f -value. An overestimating heuristic makes good nodes look worse than they are (artificially high f), so they sit deeper in the fringe while A* explores suboptimal-looking nodes with deceptively low f .
Still complete — overestimation does not affect completeness.	A* will still find <i>a</i> solution (the search is still systematic). It just may not be the <i>best</i> one.

Example — inadmissible (overestimate): Suppose $h(\text{Fagaras}) = 200$, but the true cost from Fagaras to Bucharest is $h^*(\text{Fagaras}) = 99$. Then $f(\text{Fagaras}) = 239 + 200 = 439$, while the path through Pitesti has $f(\text{Pitesti}) = 317 + 101 = 418$. The Fagaras path looks worse and is explored later. If $\text{Sibiu} \rightarrow \text{Fagaras} \rightarrow \text{Bucharest}$ (cost $239 + 99 = 338$) were actually optimal but h made it look worse, A* might return the Pitesti route (cost 418) — suboptimal.

Summary

Property	h underestimates (admissible)	h overestimates (inadmissible)
Optimality	Guaranteed ✓	Not guaranteed ×
Node expansions	More than with exact h	Unpredictable — may expand fewer nodes
Completeness	Guaranteed ✓	Guaranteed ✓
Example	$h(n) = 0$ (UCS — slow but optimal)	$h(n) > h^*(n)$ (fast but potentially wrong)
Design rule	Always make h admissible	Never intentionally overestimate

Exam key point: The **admissibility condition** $h(n) \leq h^*(n)$ for all n is the **single most important property** of a heuristic in A*. It is the necessary and sufficient condition for A* optimality (on tree-search; on graph-search, consistency/monotonicity is also required for optimality without reopening nodes). An admissible heuristic guarantees you won't miss the best path; a well-chosen admissible heuristic (close to h^*) makes the search efficient.